

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What will be printed?

```
def make_counter():
    count = 0
```

```
    def counter():
        nonlocal count
        count += 1
        return count
```

```
    return counter
```

```
a = make_counter()
b = make_counter()
```

```
print(a())
print(a())
print(b())
print(a())
```

A)1

1

1

1

B)1

2

1

3

C)0

1

0

2

D)1

2

3

4

Q2. What happens when this code runs?

```
def first():
    second()
```

```
def second():
```

```
third()
```

```
def third():
    raise RuntimeError("Failure")
```

```
try:
    first()
except RuntimeError:
    print("Handled")
```

A)NameError

B)Handled

C)TypeError

D)RuntimeError cannot propagate through functions

Q3. What does the following function return?

```
def fun():
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

A)Hello

B>Error

C)Hello None

D)None

Q4. What will happen?

```
def fun(a, b=10):
    return a + b
```

```
print(fun(b=5, a=20))
```

A)25

B)SyntaxError

C)15

D)TypeError

Q5. What is the output?

```
def fun():
    try:
        print("A")
```



```

raise ValueError
except ValueError:
    print("B")
    return 10
else:
    print("C")
    return 20
finally:
    print("D")

```

print(fun())

A)A

C

D

20

B)A

B

10

D

C)A

B

C

D

10

D)A

B

D

10

Q6. Suppose the current directory contains a file named random.py. What potential problem can occur with:

import random

A)The local file can shadow the intended standard-library module

B)random cannot be imported

C)Python automatically renames the local file

D)Python will always reject it

Q7. What happens here?

from math import sqrt

print(sqrt(36))

A)6.0

B)36

C)6

D)NameError

Q8. What is the purpose of this statement?

if __name__ == "__main__":

A)It creates the main function

B)It prevents all imports

C)It identifies code intended to run when the file is executed directly

D)It prevents syntax errors

Q9. What is the output?

def fun(x):

x += 10

return x

x = 5

y = fun(x)

print(x, y)

A)15 5

B)15 15

C)5 5

D)5 15

Q10. What will be printed?

def fun(x=[]):

x.append(10)

return x

a = fun()

b = fun()

print(a)

print(b)

A)[10]

[10]

B)[10]

[10, 10]

C)[10, 10]

[10]

D>Error

Q11. What will be printed?

try:

print(10 / 0)

except ZeroDivisionError:



```
print("A")
except Exception:
    print("B")
```

- A) A B
- B) Nothing
- C) A
- D) B

Q12. What is the major difference between the following?

```
import math
```

and

```
from math import sqrt
```

- A) First provides the module namespace; second imports sqrt directly
- B) There is no difference
- C) First imports only sqrt
- D) Both import only sqrt

Q13. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

- A) 15
- B) 5
- C) 10
- D) Error

Q14. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

- A) NameError
- B) 10
- C) 20
- D) None

Q15. What is the output?

```
def fun():
    try:
        return 10
    finally:
```

```
return 20
```

```
print(fun())
```

- A) 20
- B) RuntimeError
- C) None
- D) 10

Q16. What happens here?

```
try:
    raise ValueError("Wrong value")
except ValueError as e:
    print(e)
```

- A) Wrong Value
- B) Nothing
- C) e
- D) ValueError

Q17. What is the output?

```
def outer():
    x = 10

    def inner():
        return x + 5

    return inner
```

```
f = outer()
print(f())
```

- A) None
- B) 15
- C) NameError
- D) 10

Q18. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is

normally printed?

A)10

10

B)ImportError

C)Module loaded

10

D)Module loaded

Module loaded

10

Q19. What happens when this function is called?

```
def fun(a=10, b):
```

```
    return a + b
```

A)It executes correctly

B)TypeError

C)SyntaxError

D)NameError

Q20. What is the output?

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

A)A B

B)A

C)B

D)SyntaxError

Q21. What will be the output?

```
def fun(x):
```

```
    try:
```

```
        if x < 0:
```

```
            raise ValueError("negative")
```

```
        return x
```

```
    except ValueError:
```

```
        return 100
```

```
    finally:
```

```
        print("cleanup")
```

```
print(fun(-5))
```

A)100

cleanup

B)cleanup

-5

C)cleanup

100

D)negative

cleanup

Q22. What will be printed?

```
try:
```

```
    x = int("10")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)A C

B)B

C)B C

D)A

Q23. What will be printed?

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        print("Finally")
```

```
print(fun())
```

A)Finally only

B)10 followed by Finally

C)Finally followed by 10

D)10 only

Q24. What is the output?

```
def fun(x):
```

```
    return x * 2
```

```
a = fun
```

```
print(a(5))
```

A>Error

B)func(5)

C)10

D)5

Q25. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
```

```
    print("B")
```



If another Python program executes:

```
import test
```

what will normally be printed?

- A)A
- B)Nothing
- C)B
- D)A and B

Q26. What is the output?

```
def fun(x):  
    if x == 0:  
        return  
    return x + fun(x - 1)
```

```
print(fun(3))
```

- A)None
- B)6
- C)3
- D)5

Q27. What is the output?

```
try:  
    x = int("abc")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:  
    print("C")
```

- A)A C
- B)A
- C)B C
- D)B

Q28. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

```
fun()
```

- A)A B
- B)Nothing
- C)A
- D)B

Q29. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
            numbers)  
    )  
)
```

```
print(result)
```

- A)[4, 8]
- B)[1, 3, 5]
- C)[2, 4, 6, 8, 10]
- D)[2, 4]

Q30. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

```
variable = 10
```

```
fun()
```

- A)50
- B)NameError
- C)UnboundLocalError
- D)10

Q31. What will be printed?

```
x = 10
```

```
def outer():  
    x = 20
```

```
    def inner():  
        print(x)
```

```
    x = 30  
    inner()
```

```
outer()
```

- A)10
- B)30
- C)20
- D)NameError



Q32. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()
```

A)Only Hello

B)10

Hello

C)Error

D)x

display

Q33. Which object keeps track of modules that have already been imported during the current Python process?

A)sys.path

B)sys.modules

C)__module__

D)sys.imports

Q34. What happens?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

A)30

B)SyntaxError

C)20

D)TypeError

Q35. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went  
wrong")
```

```
try:  
    test()
```

```
except MyError as e:  
    print(type(e).__name__)
```

A)Something went wrong

B)Exception

C)MyError

D)TypeError

Q36. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

A)20

B)10

C)30

D)-10

Q37. What will be printed?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, b=20))
```

A)30

B)20

C)SyntaxError

D)TypeError

Q38. Why is

try:

```
    print(10 / 0)
```

except Exception:

```
    print("A")
```

except ZeroDivisionError:

```
    print("B")
```

codes's second handler never reached?

A)Exception is a superclass of ZeroDivisionError

B)except blocks execute randomly

C)ZeroDivisionError occurs before Exception

D)ZeroDivisionError is not an exception

Q39. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```



print(fun(3))

A)RecursionError

B)3

C)TypeError

D)6

Q40. What happens here?

```
def fun(a):  
    return a
```

print(fun())

A)None

B)NameError

C)TypeError

D)0

Q41. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal x  
        x = x + 5  
        return x
```

```
    inner()  
    return x
```

print(outer())

A)15

B)UnboundLocalError

C)10

D)5

Q42. What will be the output?

import math

print(math.sqrt(25))

A)5.0

B)sqrt(25)

C)5

D)25

Q43. What will be the output?

x = 10

```
def fun():  
    print(x)
```

x = 20

fun()

A)UnboundLocalError

B)10

C)20

D)NameError

Q44. What will be printed?

```
class InvalidAgeError(Exception):  
    pass
```

```
def check_age(age):
```

```
    if age < 18:
```

```
        raise InvalidAgeError("Age is  
too low")
```

```
    return "Allowed"
```

```
try:
```

```
    print(check_age(15))
```

```
except InvalidAgeError as e:
```

```
    print(type(e).__name__)
```

```
    print(e)
```

A)Nothing, because custom exceptions cannot be caught

B)InvalidAgeError

15

C)InvalidAgeError

Age is too low

D)Exception

Age is too low

Q45. What will be printed?

from functools import reduce

numbers = [1, 2, 3, 4]

```
result = reduce(  
    lambda a, b: a * b,
```

```
    numbers  
)
```

print(result)

A)24

B)0

C)9



D)10

Q46. What will happen?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal y
```

```
        y = 20
```

```
    inner()
```

```
outer()
```

A)SyntaxError

B)10

C)20

D)NameError

Q47. What will normally happen?

```
import math
```

```
import math
```

A)The module is normally loaded and executed twice

B)ImportError

C)SyntaxError

D)The module is normally loaded once and reused

Q48. What is the output?

```
try:
```

```
    raise ValueError
```

```
except ValueError:
```

```
    print("A")
```

```
    raise
```

A)A, followed by propagation of the same exception

B)A, and execution ends normally

C)No Output

D)B

Q49. What will be the output?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        global x
```

```
x = 30
```

```
inner()
```

```
print(x)
```

```
outer()
```

```
print(x)
```

A)20

10

B)30

20

C)30

30

D)20

30

**Q50. if test.py has
print("A")**

```
if __name__ == "__main__":
```

```
    print("B")
```

**If test.py is executed directly, what
will be printed?**

A)Nothing

B)A

C)B

D)A followed by B

